

Package ‘PXN’

September 17, 2025

Type Package

Title Cross-Platform Normalization and Prediction via PPCA (PXN)

Version 0.1.0

Description Implements a probabilistic principal component analysis (PPCA) based model for normalizing data from multiple platforms. This package is particularly useful for integrating and normalizing high-dimensional biological data, such as gene expression data from different microarray or sequencing platforms. It provides functions for model training, prediction, and evaluation, including cross-validation and parameter estimation.

URL <https://zhiningsui.github.io/PXN>, <https://github.com/zhiningsui/PXN>

BugReports <https://github.com/zhiningsui/PXN/issues>

License MIT + file LICENSE

Encoding UTF-8

LazyData true

LazyDataCompression xz

Imports Rfast,
stats,
graphics,
limma,
matrixStats,
grDevices

Roxygen list(markdown = TRUE)

RoxygenNote 7.3.2

Depends R (>= 4.1)

Suggests knitr, rmarkdown, ggplot2, testthat (>= 3.0.0), covr, kableExtra

VignetteBuilder knitr

R topics documented:

ameans	2
BetaEst	3
CV.InitEst	4
CV.InitEst.parallel	5
cv.split	6
epca	7

fisher	8
fsvd	9
fsvd0	9
GDfun	10
getSigmaY	12
gppca	13
gradLoglik	14
InitEst	15
InitEst.LargeSTD	17
limma	18
loglik	19
ModIntegrate	21
mplot	22
MSE.SigmaY	23
MSE1	24
myboxplot	25
Prediction	26
predict_MM	27
rhat	28
rowlm	29
rsolve2	30
sim_covars	31
sim_expres	32
tr	33
vec	33
winsor	34
Ycircbar2Cov	35

ameans

Fast Marginal Mean over an Array Dimension

Description

Computes the marginal means over a specified dimension of a multi-dimensional array. This is a faster alternative to `apply(an_array, i, mean)` when `i` is a single dimension.

Usage

```
ameans(a, i)
```

Arguments

`a` A numeric array of arbitrary dimension.

`i` Integer. The dimension over which to compute the marginal means.

Details

This function reorders the array dimensions using `aperm()` to bring the target dimension to the end, and then efficiently applies `rowMeans()` to compute marginal means. It is especially efficient for large arrays compared to the base `apply()` function.

Value

A numeric array with the i -th dimension collapsed via averaging.

See Also

[apply](#), [aperm](#), [rowMeans](#)

Examples

```
x <- array(rnorm(2 * 3 * 4), dim = c(2, 3, 4))
apply(x, c(1, 3), mean) # base R version
ameans(x, 2)           # faster version over the second dimension
```

BetaEst	<i>Quasi-MLE Estimation of Covariate Effects Beta</i>
---------	---

Description

Computes the quasi-maximum likelihood estimator (quasi-MLE) of the covariate effect matrix Beta in a latent expression model with stacked domain data. If no covariates are provided ($X = \text{NULL}$), returns `NULL`.

Usage

```
BetaEst(X, Y, b1, K, b1_min = 0.1)
```

Arguments

<code>X</code>	A numeric covariate matrix. If <code>NULL</code> , no covariate adjustment is performed.
<code>Y</code>	A numeric matrix of observed stacked outcomes, with $m \times K$ rows and N columns.
<code>b1</code>	A numeric vector of slope parameters used to scale <code>Y</code> . Length must be divisible by <code>K</code> .
<code>K</code>	Integer. Number of domains (blocks) stacked in <code>Y</code> .
<code>b1_min</code>	Numeric. Minimum allowed value for <code>b1</code> to avoid division by zero. Defaults to <code>0.1</code> .

Details

- If `X` is not `NULL`, the function scales and centers `Y`, computes averaged residuals across domains, and solves a stabilized linear system to estimate Beta.
- Stability is ensured by bounding the minimum `b1` values and using a robust pseudo-inverse `rsolve2()`.

Value

If `X` is provided, returns a numeric matrix estimating Beta (effect of covariates on latent variables). Otherwise, returns `NULL`.

See Also

[rsolve2](#), [ameans](#)

Examples

```
## Not run:
set.seed(123)
Y <- matrix(rnorm(1000), nrow = 20) # stacked domains
X <- matrix(rnorm(50 * 3), nrow = 50) # 3 covariates
b1 <- runif(20, 0.5, 2)
Beta <- BetaEst(X = X, Y = Y, b1 = b1, K = 4)

## End(Not run)
```

CV.InitEst

Cross-Validation for Selecting Optimal Latent Dimension L

Description

Performs K-fold cross-validation across a grid of candidate latent dimensions (L) to select the best number of latent factors for initializing the PPCA-Xnorm model using InitEst().

Usage

```
CV.InitEst(X, Y, K, folds, k.source, k.target, Ls = 1:10, ...)
```

Arguments

X	Optional covariate matrix. If no covariates are used, set X = NULL.
Y	A numeric matrix of observed stacked outcomes, with $m \times K$ rows and n columns.
K	Integer. Number of domains/platforms.
folds	A list of test-set column indices for each CV fold (e.g., output of <code>cv.split()</code>).
k.source	Integer. Index (1-based) of the source platform within the stacked rows of Y.
k.target	Integer. Index (1-based) of the target platform within the stacked rows of Y.
Ls	Integer vector. Candidate grid of latent dimensions L to evaluate. Defaults to 1:10.
...	Additional arguments passed to InitEst() and Prediction().

Details

- For each L and fold, InitEst() is fitted on the training set, predictions are made on the test set using Prediction(), and mean squared error (MSE) is evaluated.
- The optimal Lstar is chosen to minimize the mean MSE across folds.
- If covariates are present, they are split accordingly between training and testing.

Value

A list containing:

- MSE: A matrix of mean squared errors, with rows corresponding to each candidate L and columns to CV folds.
- Lstar: The value of L that minimizes the average MSE across folds.

See Also

[InitEst](#), [Prediction](#), [cv.split](#), [MSE1](#)

Examples

```
## Not run:
set.seed(123)
Y <- matrix(rnorm(20 * 50), nrow = 20)
folds <- cv.split(ncol(Y), n.folds = 5)
res <- CV.InitEst(X = NULL, Y = Y, K = 4, folds = folds, k.source = 1, k.target = 2)
res$Lstar

## End(Not run)
```

CV.InitEst.parallel *Cross-Validation for Selecting Optimal Latent Dimension L*

Description

Performs K-fold cross-validation across a grid of candidate latent dimensions (L) to select the best number of latent factors for initializing the PPCA-Xnorm model using InitEst().

Usage

```
CV.InitEst.parallel(
  X,
  Y,
  K,
  folds,
  k.source,
  k.target,
  Ls = 1:10,
  cores = 1,
  ...
)
```

Arguments

X	Optional covariate matrix. If no covariates are used, set X = NULL.
Y	A numeric matrix of observed stacked outcomes, with $m \times K$ rows and n columns.
K	Integer. Number of domains/platforms.
folds	A list of test-set column indices for each CV fold (e.g., output of <code>cv.split()</code>).

k.source	Integer. Index (1-based) of the source platform within the stacked rows of Y.
k.target	Integer. Index (1-based) of the target platform within the stacked rows of Y.
Ls	Integer vector. Candidate grid of latent dimensions L to evaluate. Defaults to 1:10.
cores	Integer. The number of CPU cores to use for parallel processing. Defaults to 1.
...	Additional arguments passed to InitEst() and Prediction().

Value

A list containing:

- MSE: A matrix of mean squared errors, with rows corresponding to each candidate L and columns to CV folds.
- Lstar: The value of L that minimizes the average MSE across folds.

cv.split	<i>Generate Cross-Validation Splits with Approximately Equal Fold Sizes</i>
----------	---

Description

Creates a list of indices that partition 1:N into approximately equal-sized subsets for K-fold cross-validation. The function ensures folds are randomly ordered and differ in size by at most one.

Usage

```
cv.split(N, n.folds = 5)
```

Arguments

N	Integer. Total number of samples to be split.
n.folds	Integer. Number of cross-validation folds. Defaults to 5.

Details

The function ensures:

- Fold sizes differ by at most one.
- Samples are randomly shuffled using [sample](#).

Value

A list of length n.folds, where each element is an integer vector of indices for one fold.

See Also

[sample](#) for random permutation.

Examples

```
set.seed(123)
folds <- cv.split(23, n.folds = 5)
sapply(folds, length) # Check fold sizes (e.g., 5, 5, 5, 4, 4)
unlist(folds)         # Verify all indices 1:23 are included
```

epca

*Efficient Principal Component Analysis (EPCA)***Description**

A fast and memory-efficient PCA routine compatible with and faster than `hd.eigen()` in the **Rfast** package. Automatically handles wide and tall matrices. Returns scaled eigenvalues, principal components, scores (optional), and cumulative variance proportions. The function is especially useful for high-dimensional data where standard PCA methods are too slow or memory-intensive.

Usage

```
epca(
  x,
  center = TRUE,
  scale = FALSE,
  retx = TRUE,
  k = min(nrow(x), ncol(x)),
  tol = 1e-06
)
```

Arguments

<code>x</code>	A numeric matrix or data frame. Rows represent observations and columns represent variables.
<code>center</code>	Logical. Should the variables be centered to have zero mean? Defaults to TRUE.
<code>scale</code>	Logical. Should the variables be scaled to have unit standard deviation? Defaults to FALSE.
<code>retx</code>	Logical. Should the principal component scores be returned? Defaults to TRUE.
<code>k</code>	Integer. Number of principal components to compute. Defaults to <code>min(nrow(x), ncol(x))</code> .
<code>tol</code>	Numeric. Eigenvalue tolerance. Components with eigenvalues below this threshold are discarded. Defaults to <code>1e-6</code> .

Details

This implementation computes the SVD or eigendecomposition of the (possibly transposed) covariance matrix, depending on the dimensionality of `x`. For wide matrices ($n < m$), it avoids computing the full covariance matrix to save time and memory. Eigen decomposition is preferred for numerical stability and speed.

Value

A list containing:

`values` A numeric vector of eigenvalues scaled by $\sqrt{1 / (n - 1)}$.

`vectors` A matrix of principal component directions (eigenvectors). Columns are named "PC1", "PC2", etc.

`x` (If `retx = TRUE`) The principal component scores matrix.

`var.props` A numeric vector of cumulative variance proportions explained by the components.

See Also

[hd.eigen](#), [prcomp](#), [eigen](#)

Examples

```
## Not run:
set.seed(123)
X <- matrix(rnorm(100 * 20), nrow = 100)
result <- epca(X)
plot(result$values, type = "b", main = "Eigenvalues")

## End(Not run)
```

fisher

Fisher's Method for Combining P-values

Description

Combines multiple independent p-values into a single global p-value using Fisher's combination test.

Usage

```
fisher(ps, pmin = 1e-06)
```

Arguments

ps	A numeric vector of individual p-values to combine.
pmin	Numeric. Minimum threshold for individual p-values to avoid taking $\log(0)$. Defaults to 1e-6.

Details

- The test statistic is computed as $S = -2 \sum \log(p_i)$, where each p_i is thresholded by pmin.
- The combined p-value is obtained by comparing S to a chi-squared distribution with $2 \times \text{length}(ps)$ degrees of freedom.
- Useful for aggregating evidence across multiple tests while controlling type I error.

Value

A numeric scalar representing the combined global p-value.

See Also

[pchisq](#)

Examples

```
## Not run:
ps <- c(0.01, 0.05, 0.10)
fisher(ps)

## End(Not run)
```

fsvd

*Fast SVD with Automatic Transposition***Description**

A fast singular value decomposition for high-dimensional matrices. Automatically transposes if rows < columns to optimize computation.

Usage

```
fsvd(X, k = min(nrow(X), ncol(X)), tol = 1e-06)
```

Arguments

X	A numeric matrix.
k	Number of singular vectors/values to retain. Defaults to <code>min(nrow(X), ncol(X))</code> .
tol	Tolerance threshold for filtering small eigenvalues. Defaults to 1e-6.

Value

A list with components d (singular values), u, and v.

fsvd0

*Core Fast SVD Computation***Description**

Performs fast SVD using eigendecomposition of $X'X$. Assumes input matrix is wide.

Usage

```
fsvd0(X, k = min(nrow(X), ncol(X)), tol = 1e-06)
```

Arguments

X	A numeric matrix.
k	Number of singular vectors/values to retain. Defaults to <code>min(nrow(X), ncol(X))</code> .
tol	Tolerance threshold for filtering small eigenvalues. Defaults to 1e-6.

Value

A list with components d (singular values), u, and v.

GDfun

*Gradient Descent for Updating b1 and Beta***Description**

Performs gradient descent optimization to update the slope parameters b_1 and covariate coefficients $Beta$ in a latent expression model. The intercepts b_0 are recomputed as row means of Y at each step. The eigenstructure parameters (U , dd , σ_{2s} , σ_{U2s}) are assumed to be fixed and estimated separately via `InitEst()`.

Usage

```
GDfun(
  X,
  Y,
  K,
  params,
  platforms = NULL,
  s0 = 0.01,
  smax = 0.5,
  sigma2.min = 0.01,
  sigma2.max = 2,
  b1_min = 0.1,
  b1.max = 5,
  beta.min = -5,
  beta.max = 5,
  max.iter = 30,
  tol = 0.1,
  verbose = FALSE
)
```

Arguments

<code>X</code>	Optional covariate matrix. If not <code>NULL</code> , must be a numeric matrix with <code>ncol(X) = ncol(Y)</code> .
<code>Y</code>	A numeric matrix of observed data with $m \times K$ rows and N columns, where m is the number of genes/features.
<code>K</code>	Integer. Number of domains/platforms.
<code>params</code>	A list containing the current parameter estimates: <code>b1</code>: Initial slope vector. <code>U</code>, <code>dd</code>: Eigenvectors and eigenvalues of the latent structure. <code>sigmaU2s</code>, <code>sigma2s</code>: Variance components.
<code>platforms</code>	Optional character vector of platform/domain names of length K .
<code>s0</code>	Numeric. Initial step size for gradient descent. Defaults to <code>0.01</code> .
<code>smax</code>	Numeric. Maximum allowed step size. Defaults to <code>0.5</code> .
<code>sigma2.min</code> , <code>sigma2.max</code>	Numeric. (Currently unused) Reserved for future extensions to variance parameter updates.

b1_min, b1_max	Numeric. Lower and upper bounds for b1. Defaults are 0.1 and 5.
beta_min, beta_max	Numeric. Lower and upper bounds for Beta entries. Defaults are -5 and 5.
max.iter	Integer. Maximum number of gradient descent iterations. Defaults to 30.
tol	Numeric. Tolerance for convergence based on log-likelihood difference. Defaults to 0.1.
verbose	Logical. If TRUE, returns intermediate results including Ycirc and Ycircbar. Defaults to FALSE.

Details

The function uses a stabilized gradient descent approach with adaptive step size control. If the log-likelihood decreases after a proposed update, the step size is halved until improvement is achieved or the maximum number of retries is reached. All updates to b1 and Beta are thresholded to specified bounds for numerical stability.

If no eigenstructure (U or dd) is available (i.e., NULL), the function skips optimization and returns the input parameters.

Value

If verbose = FALSE, returns a list containing updated params with components:

- b0, b1, Beta, U, dd, sigma2s, sigmaU2s, N, K.

If verbose = TRUE, additionally returns:

- Ycirc: Adjusted residual matrix.
- Ycircbar: Averaged residual matrix across domains.
- loglik: Final log-likelihood value.
- loglik.old: Initial log-likelihood value.
- iters: Number of iterations performed.
- history: Trajectory of log-likelihood and step size per iteration.

See Also

[gradLoglik](#), [loglik](#), [BetaEst](#), [ameans](#)

Examples

```
## Not run:
set.seed(123)
Y <- matrix(rnorm(1000), nrow = 20)
params <- list(
  b1 = runif(20, 0.5, 2),
  U = matrix(rnorm(100), nrow = 20),
  dd = runif(5, 0.1, 1),
  sigmaU2s = runif(20, 0.1, 0.5),
  sigma2s = runif(20, 0.1, 0.5)
)
res <- GDfun(X = NULL, Y = Y, K = 4, params = params, verbose = TRUE)
res$params$b1

## End(Not run)
```

getSigmaY

Compute the Covariance Matrix Σ_Y from Estimated Parameters

Description

Computes the full covariance matrix or only the diagonal (variance terms) of Σ_Y based on the estimated model parameters from PPCA-Xnorm or generalized PPCA models.

Usage

```
getSigmaY(est, var.only = FALSE)
```

Arguments

<code>est</code>	A fitted model object (e.g., output from <code>InitEst()</code> , <code>gppca()</code> , or related functions) containing at least <code>U</code> , <code>dd</code> , <code>sigmaU2s</code> , and <code>sigma2s</code> .
<code>var.only</code>	Logical. If <code>TRUE</code> , returns only the vector of marginal variances (diagonal elements of Σ_Y). Defaults to <code>FALSE</code> .

Details

- $\Sigma_Y = \text{diag}(\sigma^2 + \sigma_U^2) + AA'$, where AA' captures the shared latent structure.
- If no latent factors are available (`U = NULL` or `dd = NULL`), only the diagonal variance terms are used.
- This function automatically adapts to simple cases (e.g., pure noise models) and complex cases (low-rank plus diagonal structure).

Value

- If `var.only = TRUE`, returns a numeric vector of variances for each gene/feature.
- If `var.only = FALSE`, returns a full covariance matrix Σ_Y .

See Also

[InitEst](#), [gppca](#)

Examples

```
## Not run:
set.seed(123)
est <- list(
  U = matrix(rnorm(100), nrow = 10),
  dd = runif(10, 0.1, 1),
  sigmaU2s = runif(10, 0.05, 0.1),
  sigma2s = runif(10, 0.05, 0.1)
)
SigmaY_full <- getSigmaY(est)
SigmaY_diag <- getSigmaY(est, var.only = TRUE)

## End(Not run)
```

Description

Performs a generalized form of probabilistic PCA allowing for heterogeneous (non-scalar) noise variances across variables. The method estimates latent components and variable-specific noise variances using an iterative EM-like algorithm. It is compatible with `svd` and `prcomp` data formats and automatically handles centering and scaling. When the number of latent components is zero, the function returns noise variances only.

Usage

```
gppca(
  Y,
  center = TRUE,
  scale = FALSE,
  retx = TRUE,
  L = min(nrow(Y), ncol(Y)),
  min.sigma2 = 0.01,
  pca.tol = 1e-06,
  iter.tol = 1e-06,
  max.iter = 20
)
```

Arguments

<code>Y</code>	A numeric matrix of size $N \times m$, where rows are samples and columns are features.
<code>center</code>	Logical. Should the columns of <code>Y</code> be centered? Defaults to <code>TRUE</code> .
<code>scale</code>	Logical. Should the columns of <code>Y</code> be standardized? Defaults to <code>FALSE</code> .
<code>retx</code>	Logical. Currently unused. Kept for compatibility. Defaults to <code>TRUE</code> .
<code>L</code>	Integer. Number of latent dimensions (principal components) to extract. Defaults to <code>min(nrow(Y), ncol(Y))</code> .
<code>min.sigma2</code>	Minimum allowed residual variance per variable. Used for numerical stability. Defaults to <code>0.01</code> .
<code>pca.tol</code>	Tolerance for PCA eigenvalue thresholding. Used during initialization. Defaults to <code>1e-6</code> .
<code>iter.tol</code>	Convergence tolerance for the iterative updates. Defaults to <code>1e-6</code> .
<code>max.iter</code>	Maximum number of iterations allowed for convergence. Defaults to <code>20</code> .

Details

The algorithm starts with an initial PCA estimate, infers variable-specific noise variances, and iteratively refines both the loading matrix and noise model using generalized singular value decomposition. If `L = 0`, the function estimates only per-variable noise variances. For standardized input, the model ensures estimated components correspond to a correlation structure.

Value

A list with components:

`U` Matrix of estimated loading directions (eigenvectors of the generalized correlation matrix).

`dd` Estimated eigenvalues of the latent space (diagonal elements of the covariance matrix).

`sigma2s` Estimated residual variances for each feature (non-scalar noise).

`history` A numeric vector recording the convergence error at each iteration.

See Also

[epca](#), [prcomp](#), [svd](#)

Examples

```
## Not run:
set.seed(1)
Y <- matrix(rnorm(100 * 20), nrow = 100)
res <- gppca(Y, L = 5)
plot(res$dd, type = "b", main = "Generalized Eigenvalues")

## End(Not run)
```

gradLoglik

Gradient of the Log-Likelihood with Respect to b1

Description

Computes the gradient of the log-likelihood function with respect to the slope parameters `b1`, assuming all other parameters are either estimated via closed-form conditional MLEs (`b0` and `Beta`) or moment estimators (`U`, `dd`, `sigma2s`, and `sigmaU2s`). Designed for models of stacked domain-specific data with optional covariates.

Usage

```
gradLoglik(X, Y, K, params, b1_min = 0.1)
```

Arguments

- | | |
|---------------------|--|
| <code>X</code> | Optional covariate matrix. If there are no covariates, set both <code>X</code> and <code>Beta</code> in <code>params</code> to <code>NULL</code> . |
| <code>Y</code> | A numeric matrix of observed data with <code>m*K</code> rows and <code>N</code> columns, where <code>m</code> is the number of genes and <code>K</code> is the number of domains/platforms. |
| <code>K</code> | Integer. Number of domains (blocks) stacked in <code>Y</code> . |
| <code>params</code> | A list containing model parameters: <ul style="list-style-type: none"> • <code>b0</code>: Vector of intercepts (length <code>m</code>). • <code>b1</code>: Vector of slopes (length <code>m</code>). • <code>Beta</code>: Coefficient matrix for covariates (optional). • <code>U</code>: Eigenvector matrix for latent structure. |

- `dd`: Corresponding eigenvalues.
 - `sigmaU2s`: Latent platform-specific variances.
 - `sigma2s`: Residual variances.
- `b1_min` Numeric. Minimum allowed value for `b1` to avoid division by zero or $\log(0)$ errors. Defaults to 0.1.

Details

- Only the derivatives with respect to `b1` are computed.
- The structure of `Y` is assumed to match the stacking: `[Domain1; Domain2; ...; DomainK]`.
- Uses efficient array manipulations and precomputes cross-term matrices for gradient stability.

Value

A numeric vector of length equal to `length(b1)`, containing the gradient of the log-likelihood with respect to each slope parameter.

See Also

[loglik](#), [ameans](#), [tr](#)

Examples

```
## Not run:
# Example pseudo-code
params <- list(
  b0 = rnorm(5),
  b1 = runif(5, 0.5, 2),
  Beta = NULL,
  U = matrix(rnorm(25), nrow = 5),
  dd = runif(5, 0.1, 1),
  sigmaU2s = runif(5, 0.1, 0.5),
  sigma2s = runif(5, 0.1, 0.5)
)
Y <- matrix(rnorm(5 * 2 * 50), nrow = 10)
grad <- gradLoglik(X = NULL, Y = Y, K = 2, params = params)

## End(Not run)
```

InitEst

Initial Parameter Estimation for Cross-Platform Normalization

Description

Provides initial estimates of model parameters for cross-platform normalization or mapping models. Handles genes with both sufficient and insufficient variance separately. Acts as a wrapper around `InitEst.LargeSTD()` for genes with sufficiently large variance, and fills simple initializations for others.

Usage

```
InitEst(X, Y, K, L, min.sigma2 = 0.01, min.sigmaU2 = 0.05, p.correct = TRUE)
```

Arguments

X	Optional covariate matrix. If no covariates are used, set X = NULL.
Y	A numeric matrix of stacked domain-specific observed outcomes, with $m \times K$ rows and N columns.
K	Integer. Number of platforms (domains).
L	Integer. Number of latent factors to extract for the latent structure (U, dd).
min.sigma2	Numeric. Minimum variance threshold for retaining genes. Genes with lower variance are handled separately. Defaults to 0.01.
min.sigmaU2	Numeric. Minimum threshold for latent variance estimates. Defaults to 0.05.
p.correct	Logical. If TRUE, performs variance proportion correction for latent structure. Defaults to TRUE.

Details

- Genes with low variance (`rowVars(Y)` below `min.sigma2` across all platforms) are assigned simple defaults.
- For genes with sufficient variance, full initialization is performed using `InitEst.LargeSTD()`.
- Covariate matrix X is automatically coerced to a matrix if necessary.
- If latent factors are not identified (e.g., $L = 0$), the returned U will be NULL.

Value

A list containing the initial parameter estimates:

- `b0`: Gene-by-platform matrix of intercepts.
- `b1`: Gene-by-platform matrix of slopes.
- `Beta`: Covariate coefficient matrix (if X is provided).
- `U`: Estimated eigenvector matrix for latent structure.
- `dd`: Estimated eigenvalues (squared singular values).
- `varprops`: Cumulative variance proportions explained by latent factors.
- `sigmaU2s`: Gene-specific latent variances.
- `sigma2s`: Gene-specific residual variances.
- `N`: Number of samples.
- `K`: Number of platforms.
- `low.var.genes`: Indices of genes removed due to low variance.

See Also

[InitEst.LargeSTD](#), [BetaEst](#), [rowVars](#)

Examples

```
## Not run:
set.seed(123)
Y <- matrix(rnorm(1000), nrow = 20)
res <- InitEst(X = NULL, Y = Y, K = 4, L = 3)
names(res)

## End(Not run)
```


Description

Computes initial estimates of model parameters (b_0 , b_1 , β , U , dd , σ_{U2s} , σ_{2s}) for genes with sufficiently large variance using a stabilized two-step procedure: centering and standardization of residuals, followed by generalized probabilistic PCA (gppca) for covariance estimation. This function is intended for use inside `InitEst()` and does not handle low-variance genes.

Usage

```
InitEst.LargeSTD(
  X,
  Y,
  K,
  L,
  min.sigma2 = 0.01,
  min.sigmaU2 = 0.05,
  W = 0.5,
  p.correct = TRUE
)
```

Arguments

X	Optional covariate matrix. If no covariates are available, set <code>X = NULL</code> .
Y	A numeric matrix of observed stacked outcomes, with $m \times K$ rows and N columns.
K	Integer. Number of domains/platforms.
L	Integer. Number of latent factors to extract.
min.sigma2	Numeric. Minimum allowed residual variance per gene. Defaults to 0.01.
min.sigmaU2	Numeric. Minimum allowed latent variance per gene. Defaults to 0.05.
W	Numeric. Weight (between 0 and 1) splitting variance between latent (σ_{U2s}) and residual (σ_{2s}) components. Defaults to 0.5.
p.correct	Logical. Whether to apply degrees of freedom correction for R_0sbar after covariate regression. Defaults to TRUE.

Details

- Genes are assumed to have sufficiently large variance (standard deviation well above `min.sigma2`).
- Residuals are computed after optionally regressing out covariates using a robust hat matrix (`rhat()`).
- Standardized residuals (R_0s) are averaged across domains to form R_0sbar , which is used to estimate the latent structure.
- Uses `gppca()` to extract low-dimensional latent components.

Value

A list containing:

- `b0`: Gene-by-platform matrix of intercepts.
- `b1`: Gene-by-platform matrix of slopes.
- `Beta`: Covariate coefficient matrix (if `X` is provided).
- `U`: Estimated eigenvector matrix for shared latent structure.
- `dd`: Estimated eigenvalues.
- `varprops`: Cumulative variance proportions explained by the latent structure.
- `sigmaU2s`: Gene-specific latent variances.
- `sigma2s`: Gene-specific residual variances.

See Also

[InitEst](#), [gppca](#), [BetaEst](#), [Ycircbar2Cov](#)

Examples

```
## Not run:
set.seed(123)
Y <- matrix(rnorm(1000), nrow = 20)
res <- InitEst.LargeSTD(X = NULL, Y = Y, K = 4, L = 3)
names(res)

## End(Not run)
```

limma

Linear Modeling with Empirical Bayes Moderation (limma Wrapper)

Description

Fits gene-wise linear models using covariates `v` and applies empirical Bayes moderation to stabilize variance estimates. Returns estimated coefficients, t-statistics, p-values, and adjusted p-values for each covariate of interest.

Usage

```
limma(gdata, v, padj.method = "BH")
```

Arguments

<code>gdata</code>	A numeric gene expression matrix. Rows are genes/features, columns are samples.
<code>v</code>	A covariate matrix or vector, where rows correspond to samples and columns to covariates.
<code>padj.method</code>	Character string specifying the method for p-value adjustment. Defaults to "BH" (Benjamini-Hochberg).

Details

- Samples with missing values in `v` are removed prior to model fitting.
- The model fitted is: $Y = \text{Intercept} + v_1 + v_2 + \dots + \epsilon$.
- `eBayes()` from the **limma** package is used to perform empirical Bayes shrinkage of variance estimates.

Value

A `data.frame` containing:

- Estimated coefficients (`betahat.*`) for each covariate.
- Moderated t-statistics (`tstat.*`) for each covariate.
- Raw p-values (`pvals.*`) for each covariate.
- Adjusted p-values (`adjP.*`) for each covariate.

See Also

[lmFit](#), [eBayes](#)

Examples

```
## Not run:
library(limma)
set.seed(123)
gdata <- matrix(rnorm(1000), nrow = 100) # 100 genes x 10 samples
v <- matrix(rnorm(10 * 2), ncol = 2)      # 2 covariates
colnames(v) <- c("Treatment", "Batch")
res <- limma(gdata, v)
head(res)

## End(Not run)
```

Description

Computes the log-likelihood (without the -2 multiplier) for a latent variable model based on observed gene expression data, structured into K domains or platforms. The model includes affine transformations, latent correlation structure via eigen decomposition, and covariate adjustment if applicable.

Usage

```
loglik(X, Y, K, params, b1_min = 0.1, verbose = FALSE)
```

Arguments

X	Optional covariate matrix. If there are no covariates, both X and Beta in params should be set to NULL.
Y	A numeric matrix of observed data with $m \times K$ rows and N columns, where m is the number of genes and K is the number of platforms.
K	Integer. Number of platforms (or blocks) the rows of Y are divided into.
params	A list containing the model parameters: • b0: Vector of intercepts (length m). • b1: Vector of slopes (length m). • Beta: Coefficient matrix for covariates (optional). • U: Eigenvector matrix of the latent correlation structure. • dd: Corresponding eigenvalues (dd[1] equals squared singular values). • sigmaU2s: Platform-specific variance components. • sigma2s: Residual variances per gene.
b1_min	Numeric. Minimum allowed value for b1 slopes to avoid numerical instability. Defaults to 0.1.
verbose	Logical. If TRUE, returns detailed components of the likelihood calculation for debugging. Defaults to FALSE.

Details

- U and dd characterize the latent correlation structure AA' .
- The likelihood accounts for transformations, platform-specific noise variances, and optional covariates.
- Computations are stabilized by imposing minimum thresholds on b1 values.
- The function uses `ameans()` for fast marginalization across domains and assumes Y is ordered by stacked platforms.

Value

If verbose = FALSE, returns a numeric scalar representing the log-likelihood. If verbose = TRUE, returns a named vector containing intermediate quantities (logdet, Term1, Term2, Term3a, Term3b, Term3, loglik).

See Also

[ameans](#), [tr](#), [solve](#)

Examples

```
## Not run:
# Example pseudo-code usage
params <- list(
  b0 = rnorm(5),
  b1 = runif(5, 0.5, 2),
  Beta = NULL,
  U = matrix(rnorm(25), nrow = 5),
  dd = runif(5, 0.1, 1),
  sigmaU2s = runif(5, 0.1, 0.5),
  sigma2s = runif(5, 0.1, 0.5)
```

```

)
Y <- matrix(rnorm(5 * 2 * 50), nrow = 10)
loglik(X = NULL, Y = Y, K = 2, params = params)

## End(Not run)

```

ModIntegrate

*Model Integration Across Multiple PPCA-XNORM Trained Models***Description**

Integrates a list of separately trained PPCA-Xnorm models (typically from different datasets or study groups) into a single unified model by combining intercepts, slopes, covariate effects, and latent structures. Shared covariance structure is re-estimated based on the averaged residuals (Y_{cirbar}) from all groups.

Usage

```
ModIntegrate(modList, Xlist = NULL, Ylist, PairList)
```

Arguments

<code>modList</code>	A list of trained PPCA-Xnorm models, where each model is a list output from GDfun .
<code>Xlist</code>	Optional list of covariate matrices corresponding to each group. If no covariates are used, set <code>Xlist = NULL</code> .
<code>Ylist</code>	A list of observed outcome matrices (one for each group), stacked by platform/domain order.
<code>PairList</code>	A list or matrix indicating the platform identities for each group, used to align and integrate b_0 and b_1 .

Details

- Intercepts (b_0) and slopes (b_1) are combined via sample-size weighted averages across groups sharing the same platform.
- Covariate effects (β) are combined via weighted averages if available.
- Shared latent structure (U , dd) is re-estimated by applying SVD to concatenated Y_{cirbar} matrices.
- Effective number of PCs (L) may decrease if singular values are numerically zero during SVD.

Value

A list containing:

- b_0 : Gene-by-platform matrix of integrated intercepts.
- b_1 : Gene-by-platform matrix of integrated slopes.
- β : Integrated covariate coefficient matrix (if covariates were provided).
- U : Integrated eigenvector matrix of the shared latent structure.

- dd: Integrated eigenvalues (latent variances).
- varprops: Cumulative variance proportions explained by the integrated latent structure.
- sigma2s: Integrated gene-specific residual variances.
- sigmaU2s: Integrated gene-specific latent variances.
- Ns: Vector of sample sizes per group.
- Ks: Vector of number of platforms per group.

See Also

[GDFun](#), [InitEst](#), [fsvd](#), [ameans](#)

Examples

```
## Not run:
# Example pseudo-usage
set.seed(123)
mod1 <- GDFun(NULL, matrix(rnorm(20 * 50), nrow = 20), K = 2, params = list(b1 = rep(1, 20), U = NULL, dd = NULL, s
mod2 <- GDFun(NULL, matrix(rnorm(20 * 60), nrow = 20), K = 2, params = list(b1 = rep(1, 20), U = NULL, dd = NULL, s
modList <- list(mod1$params, mod2$params)
PairList <- list(c("Platform1", "Platform2"), c("Platform3", "Platform4"))
Ylist <- list(matrix(rnorm(20 * 50), nrow = 20), matrix(rnorm(20 * 60), nrow = 20))
res <- ModIntegrate(modList, Xlist = NULL, Ylist = Ylist, PairList = PairList)
names(res)

## End(Not run)
```

mplot

Matrix Image Plot

Description

Displays a grayscale image of a matrix by flipping the vertical axis, commonly used for visualizing matrices like heatmaps.

Usage

```
mplot(mat, ...)
```

Arguments

mat	A numeric matrix to visualize.
...	Additional arguments passed to <code>image()</code> .

Value

None. Produces a plot.

MSE.SigmaY	<i>Mean Squared Error (MSE) Between Estimated and Oracle Σ_Y</i>
------------	--

Description

Computes the mean squared error (MSE) between two covariance models Σ_Y , based on their low-rank and diagonal components. Designed to efficiently handle large matrices by avoiding explicit matrix reconstruction.

Usage

```
MSE.SigmaY(est, oracle)
```

Arguments

<code>est</code>	A fitted model object (e.g., output from <code>InitEst()</code> or <code>gppca()</code>) containing <code>U</code> , <code>dd</code> , <code>sigmaU2s</code> , and <code>sigma2s</code> .
<code>oracle</code>	The true model object (oracle) with the same structure (<code>U</code> , <code>dd</code> , <code>sigmaU2s</code> , and <code>sigma2s</code>).

Details

- The low-rank term is computed as AA' where $A = U\sqrt{d}$.
- The full covariance matrix Σ_Y is modeled as the sum of a low-rank term (AA') and a diagonal variance term.
- The MSE is computed as the sum of squared differences between the two low-rank structures and the two sets of variances, divided by m^2 .
- This method is memory-efficient and avoids explicitly forming large Σ_Y matrices.

Value

A numeric scalar representing the mean squared error between the estimated and oracle Σ_Y matrices, scaled by $1/m^2$.

See Also

[getSigmaY](#), [InitEst](#), [gppca](#)

Examples

```
## Not run:
set.seed(123)
est <- list(
  U = matrix(rnorm(100), nrow = 10),
  dd = runif(10, 0.1, 1),
  sigmaU2s = runif(10, 0.05, 0.1),
  sigma2s = runif(10, 0.05, 0.1)
)
oracle <- list(
  U = matrix(rnorm(100), nrow = 10),
  dd = runif(10, 0.1, 1),
```

```

sigmaU2s = runif(10, 0.05, 0.1),
sigma2s = runif(10, 0.05, 0.1)
)
MSE.SigmaY(est, oracle)

## End(Not run)

```

MSE1

*Mean Squared Error (MSE) Between Two Vectors***Description**

Computes the mean squared error (MSE) between estimated and true values, with an option for relative MSE.

Usage

```
MSE1(est, truth, relative = FALSE)
```

Arguments

<code>est</code>	A numeric vector of estimated values.
<code>truth</code>	A numeric vector of true values, of the same length as <code>est</code> .
<code>relative</code>	Logical. If TRUE, returns the relative MSE (normalized by $\text{sum}(\text{truth}^2)$). Defaults to FALSE.

Details

- If `relative = FALSE`, computes $\frac{1}{n} \sum_i (\hat{y}_i - y_i)^2$.
- If `relative = TRUE`, computes $\frac{\sum_i (\hat{y}_i - y_i)^2}{\sum_i y_i^2}$, which is scale-invariant.
- Missing values (NA) are ignored in the sum.

Value

A numeric scalar representing the MSE or relative MSE.

Examples

```

set.seed(42)
truth <- rnorm(100)
est <- truth + rnorm(100, sd = 0.1)
MSE1(est, truth)
MSE1(est, truth, relative = TRUE)

```

`myboxplot`*Enhanced Boxplot with Overlaid Jittered Points*

Description

Creates a standard boxplot with jittered individual data points overlaid for better visualization of sample distributions. Outlier symbols from the boxplot itself are suppressed to avoid redundancy.

Usage

```
myboxplot(formula, data = NULL, pch = 19, ...)
```

Arguments

<code>formula</code>	A formula specifying the grouping structure, e.g., <code>y ~ group</code> .
<code>data</code>	Optional data frame containing the variables in the formula.
<code>pch</code>	Integer or character specifying the plotting symbol for the jittered points. Defaults to 19 (solid circles).
<code>...</code>	Additional arguments passed to <code>boxplot()</code> .

Details

- Boxplot outliers are suppressed (`outpch = NA`) to focus on the jittered points.
- Jittering is applied using `method = "jitter"` to prevent overplotting of identical or similar values.
- Useful for small- to medium-sized datasets where individual points are informative.

Value

An object of class `"boxplot"`, invisibly returned (the same as `boxplot()`).

See Also

[boxplot](#), [stripchart](#)

Examples

```
## Not run:
set.seed(123)
group <- rep(c("A", "B"), each = 20)
value <- c(rnorm(20, mean = 0), rnorm(20, mean = 1))
df <- data.frame(group, value)
myboxplot(value ~ group, data = df)

## End(Not run)
```

Prediction

*Cross-Platform Prediction of Gene Expression***Description**

Predicts gene expression levels on a target platform based on source platform data using a trained MatchMixeR model. Accounts for individualized scaling, covariate adjustment, shared latent structure, and platform-specific noise.

Usage

```
Prediction(
  Ysource,
  X,
  trained.model,
  k.source = 1,
  k.target = 2,
  b1_min = 0.02,
  min.sigmaU2 = 0.05,
  verbose = FALSE
)
```

Arguments

Ysource	A numeric matrix of source platform expression data. Rows are genes/features, and columns are samples.
X	Optional covariate matrix for the source data. If no covariates are available, set X = NULL.
trained.model	A list containing trained model parameters, typically produced by MatchMixeR gradient descent or estimation functions: • b0: Matrix of intercepts (genes $\times$ platforms). • b1: Matrix of slopes (genes $\times$ platforms). • Beta: Covariate coefficients (optional). • U: Eigenvectors of shared latent structure. • dd: Corresponding eigenvalues (singular values squared). • sigmaU2s: Gene-specific latent variances. • sigma2s: Gene-specific residual variances.
k.source	Integer. Index (1-based) of the source platform in b0 and b1. Defaults to 1.
k.target	Integer. Index (1-based) of the target platform. Defaults to 2.
b1_min	Numeric. Minimum threshold for slope parameters b1. Genes with slopes smaller than this value are treated with a simplified prediction. Defaults to 0.02.
min.sigmaU2	Numeric. Minimum threshold for latent variance sigmaU2s. Defaults to 0.05.
verbose	Logical. If TRUE, returns detailed prediction components. Defaults to FALSE.

Details

- Genes with very small source slopes (b1k) are predicted purely based on covariate effects and intercepts.
- Otherwise, the prediction combines gene-specific information (GSI) and shared information (SI) derived from the latent structure (U and dd).
- If missing values (NA) are present in Ysource, the function will stop. Preprocessing with winsor() is recommended.

Value

If verbose = FALSE, returns a numeric matrix Yhat of predicted expressions on the target platform.
If verbose = TRUE, returns a list containing:

- Yhat: Predicted target expression matrix.
- muk2: Adjusted covariate contribution on the target platform.
- GSI: Gene-specific information contribution.
- SI: Shared latent information contribution.
- ids0: Indices of genes where simplified prediction was applied.

See Also

[GDFun](#), [winsor](#)

Examples

```
## Not run:
# Example pseudo-usage
set.seed(123)
Ysource <- matrix(rnorm(20 * 5), nrow = 20) # 20 genes x 5 samples
trained.model <- list(
  b0 = matrix(rnorm(40), nrow = 20),
  b1 = matrix(runif(40, 0.5, 2), nrow = 20),
  Beta = matrix(rnorm(20 * 2), nrow = 20),
  U = matrix(rnorm(20 * 5), nrow = 20),
  dd = runif(5, 0.1, 1),
  sigmaU2s = runif(20, 0.05, 0.2),
  sigma2s = runif(20, 0.05, 0.2)
)
X <- matrix(rnorm(2 * 5), nrow = 2)
Ypred <- Prediction(Ysource, X, trained.model, k.source = 1, k.target = 2)

## End(Not run)
```

predict_MM

Predict Target Values Using a Trained MatchMixeR Model

Description

Applies a trained multiplicative mapping model from the MatchMixeR framework to source data Ysource to generate predicted target values. Each feature (row) in Ysource is transformed by a learned slope and intercept.

Usage

```
predict_MM(Ysource, trained.model)
```

Arguments

<code>Ysource</code>	A numeric matrix of source data. Rows correspond to features (e.g., genes), and columns to samples.
<code>trained.model</code>	A fitted model object from MatchMixeR, which must contain a matrix <code>betamat</code> with named columns "Intercept" and "Slope" representing the learned coefficients per row.

Details

For each row i , the predicted output is computed as:

$$Y_{i,\cdot}^{\text{target}} = \text{Intercept}_i + \text{Slope}_i \times Y_{i,\cdot}^{\text{source}}$$

The function uses `sweep()` to efficiently apply row-wise multiplicative transformations.

Value

A numeric matrix of the same dimensions as `Ysource`, representing the predicted target values.

See Also

[lm](#), [normalize.crossplatform](#)

Examples

```
set.seed(1)
Ysource <- matrix(rnorm(20), nrow = 4)
trained.model <- list(betamat = cbind(Intercept = rnorm(4), Slope = runif(4, 0.5, 1.5)))
Ytarget <- predict_MM(Ysource, trained.model)
```

rhat

Robust Hat Matrix Computation via Truncated SVD

Description

Computes a numerically stable approximation of the projection (hat) matrix from a design matrix X using truncated singular value decomposition (SVD). This method avoids instability due to near-singular or collinear columns by discarding small singular values below a specified threshold.

Usage

```
rhat(X, d.prop = 1e-06, dmin = 1e-09)
```

Arguments

<code>X</code>	A numeric matrix (design matrix), typically with rows representing observations and columns representing predictors.
<code>d.prop</code>	Numeric. Relative threshold for singular value truncation. Defaults to 1e-6. Threshold is computed as $\max(\text{sum}(d) * d.\text{prop}, dmin)$.
<code>dmin</code>	Numeric. Absolute minimum singular value threshold. Defaults to 1e-9.

Details

The function uses the left singular vectors U corresponding to retained singular values d to compute the hat matrix as $H = UU^\top$. This approach avoids explicit inversion and stabilizes under collinearity or rank deficiency.

Value

A numeric matrix of size $\text{nrow}(X)$ by $\text{nrow}(X)$ representing the approximate hat (projection) matrix. If all singular values fall below the threshold, returns a zero matrix.

See Also

[lm](#), [svd](#)

Examples

```
X <- matrix(rnorm(100 * 10), nrow = 100)
H <- rhat(X)
image(H)
```

rowlm

*Row-wise Linear Modeling (Affine-Invariant Alternative to limma)***Description**

Fits ordinary least squares (OLS) linear models row-by-row across high-throughput data (e.g., gene expression), manually computing coefficients, standard errors, t-statistics, p-values, and adjusted p-values. Unlike `limma()`, this implementation is strictly invariant and equivariant to affine transformations of Y .

Usage

```
rowlm(gdata, v, padj.method = "BH")
```

Arguments

<code>gdata</code>	A numeric matrix of observations. Rows are features (e.g., genes), columns are samples.
<code>v</code>	A covariate matrix or vector. Rows correspond to samples, columns correspond to covariates.
<code>padj.method</code>	Character string specifying the method for p-value adjustment. Defaults to "BH" (Benjamini-Hochberg).

Details

- Samples with missing covariate values are removed before model fitting.
- An intercept is automatically included in the design matrix.
- Manual matrix-based calculation ensures strict affine invariance of the fitted statistics.
- P-values are based on the classical t-distribution with $n - p$ degrees of freedom.

Value

A `data.frame` containing:

- Estimated coefficients (`betahat.*`) for each covariate.
- Raw t-statistics (`tstat.*`) for each covariate.
- Raw two-sided p-values (`pvals.*`) for each covariate.
- Adjusted p-values (`adjP.*`) for each covariate.

See Also

[limma](#), [lm](#), [p.adjust](#)

Examples

```
## Not run:
set.seed(123)
gdata <- matrix(rnorm(1000), nrow = 100) # 100 genes x 10 samples
v <- matrix(rnorm(10 * 2), ncol = 2)      # 2 covariates
colnames(v) <- c("Treatment", "Batch")
res <- rowlm(gdata, v)
head(res)

## End(Not run)
```

rsolve2

Robust Inversion of $X^T X$ for Tall Matrices

Description

Computes a regularized inverse of the Gram matrix $X^T X$ using truncated singular value decomposition (SVD). Designed for use with tall, thin matrices ($n \gg p$), the method stabilizes inversion in the presence of near-collinearity or small singular values, which commonly arise in regression problems.

Usage

```
rsolve2(X, d.prop = 1e-06, dmin = 1e-09, dmax = 1e+09)
```

Arguments

<code>X</code>	A numeric matrix of dimension $n \times p$, where $n \gg p$. Must not be a vector or data frame.
<code>d.prop</code>	Numeric. Proportional threshold for regularizing small singular values. Defaults to $1e-6$.
<code>dmin</code>	Numeric. Minimum allowed singular value threshold. Defaults to $1e-9$.
<code>dmax</code>	Numeric. Maximum allowed singular value threshold (for stability). Defaults to $1e9$.

Details

- If `X` is a row or column vector, closed-form stabilized solutions are computed directly.
- For full-rank matrices, SVD-based inversion with regularized singular values is applied.
- The function avoids division by small or zero singular values by applying a threshold capped between `dmin` and `dmax`.

Value

A numeric matrix of dimension $p \times p$, representing the robust inverse of $X^T X$. If the matrix is nearly singular, regularization ensures a stable estimate.

See Also

[solve](#), [rhat](#), [svd](#)

Examples

```
X <- matrix(rnorm(100 * 5), nrow = 100)
inv_xtx <- rsolve2(X)
all.equal(inv_xtx, solve(crossprod(X)), tolerance = 1e-3) # Should be close
```

`sim_covars`

Simulation covariate matrices aligned to `sim_expres`

Description

`sim_covars` contains covariate matrices that column-align with the expression matrices in [sim_expres](#). Each element is a small $p \times n$ numeric matrix (in the shipped data $p = 3$) whose rows are covariates (see `rownames()`) and whose columns are samples in the matching expression matrix.

Usage

```
data(sim_covars)
```

Format

A named list of length 3:

AB Numeric matrix $p \times n$; columns match `sim_expres$AB`.

BCD Numeric matrix $p \times n$; columns match `sim_expres$BCD`.

EF Numeric matrix $p \times n$; columns match `sim_expres$EF`.

Details

The first row is often an intercept and remaining rows are simulated covariates (e.g., clinical or technical variables). Use `rownames(sim_covars$AB)` to see the covariate labels. Before model fitting, ensure there are no missing values and that the column order matches the corresponding `sim_expres` matrix.

See Also

[sim_expres](#), [InitEst](#), [GDFun](#), [Prediction](#), [ModIntegrate](#)

Examples

```
data(sim_expres); data(sim_covars)
stopifnot(ncol(sim_covars$AB) == ncol(sim_expres$AB))
rownames(sim_covars$AB)      # covariate names

## Minimal dimensions check for all sets
lapply(sim_covars, dim)
```

sim_expres

Simulation expression matrices for PXN

Description

`sim_expres` contains simulated gene-expression matrices used in the PPCAXNORM vignette and examples. Each element is a matrix whose rows are genes and columns are samples, with platforms stacked by rows.

Usage

```
data(sim_expres)
```

Format

A named list of length 3:

AB Numeric matrix of dimension $2m \times n$. Stacked as `rbind(YA, YB)` (top = A, bottom = B).

BCD Numeric matrix of dimension $3m \times n$. Stacked as `rbind(YB, YC, YD)`.

EF Numeric matrix of dimension $2m \times n$. Stacked as `rbind(YE, YF)`.

Details

Within each element, all platforms share the same gene set and row order. Columns (samples) are specific to that dataset and align with the matching covariate matrix in [sim_covars](#) (i.e., `ncol(sim_covars$AB) == ncol(sim_expres$AB)`, etc.).

See Also

[sim_covars](#), [InitEst](#), [GDFun](#), [Prediction](#), [ModIntegrate](#)

Examples

```
data(sim_expres)
str(sim_expres, max.level = 1)
dim(sim_expres$AB)

## Split AB into source (A) and target (B) blocks
m <- nrow(sim_expres$AB) / 2
YA <- sim_expres$AB[1:m, ]
YB <- sim_expres$AB[(m + 1):(2 * m), ]
```

tr	<i>Trace of a Matrix</i>
----	--------------------------

Description

Computes the trace of a square matrix (sum of diagonal elements).

Usage

```
tr(M)
```

Arguments

M	A square numeric matrix.
---	--------------------------

Value

A numeric scalar representing the trace.

vec	<i>Vectorize a Matrix</i>
-----	---------------------------

Description

Converts a matrix to a column-major vector.

Usage

```
vec(x, mode = "any")
```

Arguments

M	A numeric matrix.
---	-------------------

Value

A numeric vector.

winsor

*Median Imputation and Winsorization by 5-Standard-Deviation Rule***Description**

Performs robust row-wise preprocessing on numeric matrix-like data by first imputing missing values using row-wise medians, and then winsorizing outliers based on a trimmed mean and standard deviation threshold. Useful for preprocessing expression data, intensity profiles, or any other high-dimensional numeric matrix.

Usage

```
winsor(data, n.std = 5, trim = 0.1, verbose = FALSE)
```

Arguments

data	A numeric matrix or data frame. Each row is treated as a feature (e.g., gene), and columns are samples.
n.std	Numeric. The number of standard deviations from the trimmed mean to use for winsorizing. Defaults to 5.
trim	Numeric. The trimming proportion (between 0 and 0.5) used to compute the trimmed mean and standard deviation. Defaults to 0.1.
verbose	Logical. Whether to print summary information about imputation and winsorization. Defaults to FALSE.

Details

- Missing values are imputed using the row-wise median (ignoring NAs).
- For each row, a trimmed mean and standard deviation are computed using central values (controlled by trim).
- Values outside the mean $\pm$ n.std $\times$ sd range are winsorized to the nearest in-range value.
- Uses fast row-wise operations from the **Rfast** package.

Value

A numeric matrix of the same dimensions as data, with missing values imputed and outliers winsorized.

See Also

[rowSort](#), [rowVars](#)

Examples

```
set.seed(42)
X <- matrix(rnorm(1000), nrow = 20)
X[sample(length(X), 10)] <- NA # introduce some NAs
X[1, 1] <- 1000 # introduce an outlier
Y <- winsor(X, verbose = TRUE)
```

Ycircbar2Cov

*Covariance Parameter Estimation from Averaged Residuals***Description**

Estimates the latent covariance structure (U , dd) and variance components (σ_{U2s} , σ_{2s}) from the matrix of domain-averaged residuals $Y_{circbar}$. Uses generalized probabilistic PCA (`gppca()`) to separate shared structure and individual noise contributions.

Usage

```
Ycircbar2Cov(Ycircbar, K, L, min.sigma2 = 0.01, min.sigmaU2 = 0.05, W = 0.5)
```

Arguments

<code>Ycircbar</code>	A numeric matrix of residuals averaged across domains, with rows as genes/features and columns as samples.
<code>K</code>	Integer. Number of domains/platforms.
<code>L</code>	Integer. Number of latent factors to extract.
<code>min.sigma2</code>	Numeric. Minimum allowed value for residual variances. Defaults to 0.01.
<code>min.sigmaU2</code>	Numeric. Minimum allowed value for latent variances. Defaults to 0.05.
<code>W</code>	Numeric. Weight parameter (between 0 and 1) controlling the division of variance between σ_{U2s} and σ_{2s} . Defaults to 0.5.

Details

- If $K = 1$ (no platform pairing), $Y_{circbar}$ is treated as standardized and `scale = TRUE` is used in `gppca()`.
- If $K \geq 2$, platform-paired adjustment is performed, and `scale = FALSE` is used.
- Variance decomposition ensures stability by enforcing minimum thresholds `min.sigma2` and `min.sigmaU2`.
- If no latent structure is detected ($U = \text{NULL}$), a simplified variance model is returned.

Value

A list containing:

- U : Estimated eigenvector matrix for shared latent structure.
- dd : Corresponding eigenvalues (latent variances).
- `varprops`: Cumulative variance proportions explained by the latent factors.
- σ_{U2s} : Estimated gene-specific latent variances.
- σ_{2s} : Estimated gene-specific residual variances.

See Also

[gppca](#)

Examples

```
## Not run:  
set.seed(123)  
Ycircbar <- matrix(rnorm(20 * 50), nrow = 20)  
cov.params <- Ycircbar2Cov(Ycircbar, K = 2, L = 3)  
names(cov.params)  
  
## End(Not run)
```